

Use1401

Programmer's Interface Library

Version 4.01

Jan 2020

Copyright © Cambridge Electronic Design Limited 1993-2010, 2019-2020

Neither the whole nor any part of the information contained in, or the product described in, this guide may be adapted or reproduced in any material form except with the prior written approval of Cambridge Electronic Design Limited.

First edition	February	1993
Second edition	October	1993
Third edition	June	1997
Fourth edition	December	1999
Fifth edition	December	2010
Sixth edition	December	2019
Seventh edition	January	2020

Published by:

Cambridge Electronic Design Limited
Science Park
Milton Road
Cambridge
CB4 0FE
UK

Telephone: Cambridge (01223) 420186
Fax: Cambridge (01223) 420488
Email: info@ced.co.uk
Web site: www.ced.co.uk

Trademarks and Trade names used in this guide are acknowledged to be the Trademarks and Trade names of their respective Companies and Corporations.

Table of Contents

The Use1401 library.....	1
Overview.....	1
Windows programs	1
Linux.....	1
Macintosh programs.....	Error! Bookmark not defined.
Changes from version 3 that impact code	1
Use1401 C/C++ support.....	3
Introduction.....	3
Function calls and U14API	3
Handles	3
Error codes.....	4
Use1401 Functions	5
Access and status functions.....	5
U14InitLib.....	5
Character transfer functions	9
Command loading functions	12
Block transfer functions	13
Test and diagnostic functions.....	17
Windows specifics.....	20
Overview.....	20
Linux specifics.....	22
Overview.....	22
Delphi and Borland Pascal specifics	23
Introduction.....	23
Types and records	23
Visual Basic specifics	24
Introduction.....	24
Python specifics	25
Introduction.....	25
Index	31

The Use1401 library

Overview This document describes the use1401 version 4 library that acts as the interface between application code and the 1401 device driver on 32 and 64-bit Windows (and Linux) operating systems. If you need to use the library on 16-bit systems you should refer to the documentation for version 3 of the library (this is no longer supported).

Windows programs The Windows library is built as a DLL, `use1432.dll`, which is provided in both 32-bit and 64-bit builds and can be used by software written in a variety of programming languages. All symbols are exported as "C", that is they do not have C++ style name mangling, and all the user functions use the `__stdcall` convention. This means that arguments are put on the stack from left to right, and the stack is cleaned up by the called function. Function names are decorated by adding `_` at the front and adding `@nn` at the back (`nn` is the number of bytes of arguments that the function expects on the stack). For example, `U14Open1401(short n1401)` is exported as `_U14Open1401@4`. To allow backwards compatibility (as far as is possible) with the previous version of the library, we also export the same symbols in undecorated form. Currently, CED only supports the use of the DLL from Microsoft C/C++, support for other languages such as Delphi 2 and Visual BASIC version 5 has been developed and tested by CED customers and is distributed by CED.

To use the library, include `use1401.h` in your source file and link with `use1432.lib`. To run the programs, `use1432.dll` must be in a suitable position for Windows to find and load it. Windows looks for DLLs in the current directory first, next in the Windows directory, then in the Windows system directories and lastly in all directories listed in the `PATH` environment variable.

Linux We do not currently support the use of the 1401 in Linux, though it very likely could be made to work. The interface library can be built and has worked in Linux. It can be built as a stand alone `.so` library, or can be linked directly into an application. It requires a Linux version of the CED 1401 USB device driver. The Linux device driver we used to develop the library is many years old and will need updates to cope with changes to Linux USB support since it was last seen working in 2012. Contact CED for more information.

Changes from version 3 that impact code You can skip this section unless you have code that used a previous version of the library.

The version 4 revisions to the library have removed all explicit 16-bit and macintosh support. They also removed the use of the `long` type from the interface due to problems of portability between Windows and Unix/Linux systems when considering 32-bit and 64-bit support. In Windows, a `long` is 32-bits regardless of 32-bit/64-bit considerations. In Linux/Unix, a `long` is 64 bits. We introduce the type `U14LONG` to be a 32-bit integer in both circumstances. We also replace `LPSTR` and `LPCSTR` with `char*` and `const char*` for reasons of portability. The following changes have been made that will require adjustments to code that worked with previous versions of the library:

1. `U14DriverVersion()` and `U14DriverType()` now require a 1401 handle, however -1 can be used to return information on the last-opened 1401.
2. Functions that returned data via a `long*` now return data via a `U14LONG*`. Other uses of `long` have changed to `int`.

3. `U14TransferEvent()` has change the type and use of the third argument.
4. All timeouts are now measured and set in milliseconds.
5. The previous version of the library did not always check the handle argument; if you pass in a bad handle you will now get an error code unless this is explicitly allowed.
6. `FAR` is no longer used to define pointers; this related to 16-bit addresses.
7. The Peek, Poke and Ramp diagnostic functions use `DWORD` arguments for the 1401 address and value fields, not `long`. This is functionally equivalent, but may cause syntax errors or warnings.

Use1401 C/C++ support

Introduction

Use1401 is a set of functions written in C to control the 1401 data acquisition system. The code may be compiled on Windows using Microsoft Visual Studio 2005 or later C/C++ compiler. It also compiles with the MinGW system. (Under Linux, it compiles under GCC.) The interface file `USE1401.H` is a set of C declarations (not C++) providing a full set of error codes, structures and functions for communication with the 1401 device. All exported functions and error codes have the prefix `U14` to indicate their origin. For use with other Windows languages, only a replacement for `USE1401.H` is required, the actual library functions are not changed.

In order to build the Use1401 library the `machine.h` file is required. This is a CED header file that mitigates some of the differences between systems. It also provides some `#defines` which are not available on one machine or the other.

The Windows version of Use1401 is intended to be built into a DLL and includes all the functions necessary to initialise the DLL at load time.

Other languages

This chapter specifically covers the use of Use1401 from programs written in C and C++, and all examples are provided for C/C++. Some examples are also given for other languages; you can get further examples of using the Use1401 library in the example programs provided along with the Use1401 libraries and include files. Because the Use1401 library is the same for all languages, this chapter contains much information which is relevant to all languages, generally only the names used to specify a type of variable or record structure change.

While most of the Use1401 functionality is available with all platforms and languages, there are a few details which only apply to one machine or the other, or vary with the programming language used. The individual function documentation gives some information on this, chapters following the function documentation give more machine and language-specific details.

Function calls and U14API

All the functions in the unit are defined with the `U14API(xxx)` macro. This macro defines the function as a `PASCAL` type function with the return type `xxx`. The macros are defined such that for:

```
U14API(short) U14Close1401(short hand);
```

the result would be suitable for the target. For example, on Windows, where we are building for use from a DLL, when you include `use1401.h`, you get:

```
short __declspec(dllimport) __stdcall U14Close1401(short hand);
```

Similar adjustments are possible for other languages.

Handles

With very few exceptions, all of the functions in the library require a 1401 handle parameter. The documentation of the individual functions assumes this parameter and do not mention it. A handle is a short integer (16 bits) which contains a value identifying the 1401 to be used, the handle value is returned by `U14Open1401` and used thereafter. If you were writing a program that used two 1401s you would call `U14Open1401` twice, receiving back two handles that would be used separately to communicate with the two 1401s opened. Once `U14Close1401` has been called the handle value is invalid and can no longer be used.

Error codes Most of the functions in Use1401 return an error code. The complete list of error codes is shown in the listing of `USE1401.H` at the end of this document. It is important that programmers use the constants provided when looking at error codes and not their values, as CED reserves the right to change the actual values used. The 'no error' value returned is also defined (as `U14ERR_NOERROR`), but it is reasonable to use the known value of zero, as below (assuming the function result is in `sErr`):

```
if (sErr == 0)           or even           if (!sErr)
```

The Use1401 library contains a function, `U14GetErrorString`, which will convert an error code returned by Use1401 into a human-readable string. You are advised to make use of this function if at all possible as it will ensure that, if new error codes are added to Use1401, your software can still display meaningful information about the error.

MINDRIVERMAJREV This constant is defined as the minimum major revision level of the device driver that this library is able to work with.

Use1401 Functions

Access and status functions

These are functions concerned with opening the 1401 for access and determining the condition of the 1401.

U14InitLib

This function is normally called automatically when the DLL or library is loaded to initialise the library. It is harmless to call it multiple times as it tests a "once-only" internal flag to prevent multiple initialisation. It returns the library revision as the major number (4) in bits 31-24, the minor revision in bits 23-16 and release numbers in bits 15-0. Major revision changes imply incompatibility, minor revisions imply new features with backwards compatibilities and release numbers imply fixes. Each field resets to 0 when a higher field changes.

```
U14API(int) U14InitLib(void);
```

U14Open1401

This function must be called before any other function is used to communicate with the 1401, it initialises the library and checks for the presence of the 1401. The return value is either the handle to be used for all further communication with the 1401, or a negative error code.

```
U14API(short) U14Open1401(short n1401);
```

The `n1401` parameter is used to select which 1401 is to be opened. If `n1401` is zero, the first available 1401 is opened. Other values of `n1401` specify the number of the 1401 to be opened, in the range 1 to 8. You can also use the value 0 to denote the first available 1401.

The `U14Open1401` function performs the following tests to determine if the 1401 is present and ready for use:

- It checks that the 1401 device driver is present.
- The 1401 device driver input and output buffers are emptied.
- Resets the 1401 using `U14Reset1401`, only if necessary.
- It requests the device driver revision level and fails if it is too old to be used.
- The 1401 status is requested from the driver.

A typical program to use the 1401 starts and finishes with code which resembles the following:

```
#include <use1401.h>
int main( )
{
    short sErr, hand;
    sErr = U14Open1401(0);
    if (sErr >= 0) /* all ok so carry on */
    {
        hand = sErr; /* save the handle FOR use */
        ... /* Use the 1401 as required */
        sErr = U14Close1401(hand); /* check for close errors */
    }
    else
        return(1); /* return error if fails */
}
```

U14Close1401

This function should be called when an application has finished using the 1401. Once this function has been called, the 1401 can no longer be used and is available for use by other applications. See `U14Open1401` for an example of its use.

```
U14API(short) U14Close1401(short hand);
```

The return value is zero or an error code.

U14Reset1401

This is used to test for a 1401. It will try to do a quick check if 1401 was OK the last time it was asked, and there is no DMA in progress and the character buffers are empty and `U14ForceReset()` has not been called. If the quick check shows that the 1401 state is still OK and the 1401 type has not changed, then there is nothing to be done, otherwise the 1401 is forced to perform a hardware reset, which may trigger a full self-test cycle if the 1401 is not in the powered up state.

If a reset is required, this will stop any running commands and flush any input or output from the I/O buffers. Any DMA in progress is stopped. All loaded commands remain intact within the 1401 unless a self-test is triggered. You can use this to get the 1401 out of a "hung" situation (for example when a dedicated 1401 command is waiting for a trigger which never comes), or for general-purpose initialisation.

```
U14API(short) U14Reset1401(short hand);
```

The return value is zero or a negative error code.

U14ForceReset

Call this before `U14Reset1401()` to force a hardware reset of the 1401.

```
U14API(short) U14Reset1401(short hand);
```

The return value is `U14ERR_NOERROR` unless the handle passed is bad.

U14TypeOf1401

This function is used to determine the type of 1401 in use. It will only return a valid result after `U14Open1401` has been used successfully.

```
U14API(short) U14TypeOf1401(short hand);
```

The return value is a positive 1401 type code as defined in `use1401.h` or `U14TYPEUNKNOWN`. It is possible for this library to open newer 1401s than it was built for as long as the device driver can cope with them as modern 1401s are designed to be backwards compatible.

U14NameOf1401

This function is used to get a string holding the name of the 1401 in use. It will only return a valid result after `U14Open1401` has been used successfully.

```
U14API(short) U14nameOf1401(short hand, char *pBuf, WORD wMax);
```

bBuf Points to a character array with space to hold the longest 1401 name (at least 16 characters is required). The longest current name is "Power1401 mk II". In Windows, if `_UNICODE` is defined, the library is built with this parameter being a `wchar_t*`.

wMax Size of the character buffer including space for a terminating 0. Longer names will be truncated.

The return value U14ERR_NOERROR unless a bad handle is used.

U14StateOf1401

This function returns a code indicating the state of the 1401. The state information is updated when the 1401 is opened, or whenever U14Reset1401 is used. The return values are standard error codes.

```
U14API (short) U14StateOf1401 (short hand);
```

The most likely return values are U14ERR_NOERROR (indicating all is OK) or:

U14ERR_OFF	The 1401 is turned off (ISA and PCI drivers only)
U14ERR_NC	The 1401 is not connected (ISA and PCI drivers only)
U14ERR_ILL	The 1401 is unwell (ISA and PCI drivers only)
U14ERR_NO1401DRIV	The 1401 device driver is not loaded
U14ERR_DRIVTOOOLD	The 1401 device driver is too old
U14ERR_NOIF	There is no interface card (ISA driver only)
U14ERR_TIME	The 1401 timed out
U14ERR_BADSW	The interface card switches are set badly (ISA driver only)
U14ERR_LOCKFAIL	Memory locking for block transfers failed
U14ERR_NOINT	Could not get the interrupt channel wanted (ISA only)
U14ERR_INUSE	The 1401 is in use by another application
U14ERR_NODMA	DMA was wanted but could not get the channel (results in programmed transfers being used)

Other values are possible if there is a problem communicating with the device driver.

U14GetUserMemorySize

This function stores the 1401 user memory size, in bytes, in the location pointed to by the parameter. The 1401 user memory is the memory available for use by 1401 commands, it runs from address zero to (size-1). Returns zero or an error code.

```
U14API (short) U14GetUserMemorySize (short hand, DWORD *pSize);
```

pSize points to a long that will be updated with the memory size.

U14LastErrCode

This function returns the last error code from a call to the library for the specified 1401. This is really only useful in Windows where there is an extra layer of error possible due to communications between the library and the device driver that do not occur in Linux.

```
U14API (short) U14LastErrCode (short hand);
```

U14GetErrorString

This function converts a Use1401 error code to a human-readable string.

```
U14API (void) U14GetErrorString (short sErr, char *pStr, WORD wMax);
```

pStr Pointer to a character array in which to store the error description.

wMax The maximum number of characters the buffer can hold including a terminating null character.

U14DriverVersion

This function returns the driver version. The high word of the function result is the major revision level and the low word is the minor revision level. If the high word of the result (the major revision level) is less than the constant `MINDRIVERMAJORREV` then `U14Open1401` will always fail. If `hand` is -1, the version of the last opened 1401 is reported.

```
U14API(long) U14DriverVersion( short hand );
```

U14DriverType

This function returns flags indicating the type of 1401 driver and 1401 interface card in use. The return value is a negative error code, or a value indicating the type of device driver: 0 ISA, 1 PCI, 2 USB. A handle of -1 is allowed to report the type of the last opened driver.

```
U14API(long) U14DriverType( void );
```

U14TransferFlags

This function returns flags providing information on the block transfer and other capabilities available. Because the 1401 support software does a good job of adjusting to the capabilities available, most applications will operate identically regardless of the flag values, but there are occasions where this information is needed. A negative error code can be returned.

```
U14API(short) U14TransferFlags(short hand);
```

The value returned will have bits set to indicate various capabilities. There are defined constants in `USE1401.H` set to the values of these bits:

Flag name	bit	Function
<code>U14TF_USEDMA</code>	0	Set if DMA is used in block transfers. Clear if programmed I/O will be used.
<code>U14TF_MULTIA</code>	1	This bit is set if multiple transfer areas can be used. If clear, then only transfer area zero is available.
<code>U14TF_FIFO</code>	2	This bit is set if an ISA interface card with FIFO buffer and demand-mode DMA capabilities is in use.
<code>U14TF_NOTIFY</code>	4	The driver supports <code>U14SetTransferEvent</code> .
<code>U14TF_CIRCCTH</code>	7	Set if the driver supports circular transfers to the host.
<code>U14TF_DIAG</code>	8	Set if the driver supports the diagnostics and debug functions.

U14MonitorRev

This function returns the 1401 monitor revision as the low part of the monitor revision level plus the high part of the revision times 1000. For example if a micro1401 monitor ROM were revision 3.13, this function would return 3013.

```
U14API(long) U14MonitorRev(short hand);
```

Character transfer functions

These functions are all concerned with the transfer of text characters between the 1401 and the host computer. They include functions to send and receive strings, testing the state of character transfers and controlling timeouts.

All character communication with the 1401 is buffered (the characters are stored in arrays both before and after transmission) to allow for efficient communication and to avoid an application hanging while it waits for a string to be transmitted. A number of the functions have an effect on these buffers.

U14SendString

This function is used to send a string to the 1401. If the buffers are full it waits (using the timeout) until space is available. The string must be terminated 'C-style', with a zero character. No semi-colon, carriage return or other terminator is appended to the string, so you will normally have to take care to use properly terminated strings. The return value is zero or an error code.

```
U14API(short) U14SendString(short hand, const char *pStr);
```

pStr Pointer to the string to be sent.

For example:

```
short sErr = U14SendString(hand, "ERR;"); /* ask for 1401 errors */
if (sErr == 0)                             /* If the string went OK */
{
    U14LONG alVal[2];
    sErr = U14LongsFrom1401(hand, alVal, 2);
    if (sErr != 2)                          /* Check we got two numbers back */
        ...
}
```

U14GetString

This function reads the next line of input from the 1401 into a string variable, terminated with a NULL character. In normal circumstances, the last character in the string will be the terminating carriage-return. If a line of input is not immediately available, the function waits for data to arrive until it times out. It stops reading when a carriage return is encountered or when the maximum number of characters have been read. If there are several replies in the input buffer, several calls to U14GetString are needed to read them all. All commas in the string are converted to spaces.

```
U14API(short) U14GetString(short hand, char *pStr, WORD wMax);
```

pStr Pointer to a character array in which to store the data.

wMax The maximum number of characters the buffer can hold including the terminating null character. The function stops reading when this number of characters have been read. The function fails if the maximum length is less than 2 as this would leave no room for any characters.

U14LongsFrom1401

This routine waits for a period defined by U14SetTimeout while trying to get a reply string from the 1401. If a reply is received it attempts to convert the string into longs using U14StrToLongs.

```
U14API(short) U14LongsFrom1401(short hand, U14LONG *palNums,
                                short sMax);
```

palNums Pointer to an array of longs to be filled in.

sMax The size of the array.

The return value is the number of values retrieved or an error code. If no values could be retrieved from the reply string, the function returns zero. For an example of use, see U14SendString.

U14StrToLongs

This is a utility routine which takes a pointer to a string and a pointer to an array of longs. The string is converted into as many longs as fit into the array, stopping when data from the string is used up. This function is used internally by U14LongsFrom1401.

```
U14API(short) U14StrToLongs(const char *pStr, U14LONG *palNums,
                           short sMax);
```

pStr Pointer to null-terminated string containing text.

palNums Pointer to an array of longs to be filled in.

sMax The size of the array.

The return value is the number of longs placed in the array or a negative error code. If no values could be retrieved from the string, the function returns zero.

U14SendChar

This function sends the single character cChar to the 1401, waiting (with timeouts) if the buffer is full. Returns zero or an error code if timed out or a bad handle.

```
U14API(short) U14SendChar(short hand, char cChar);
```

U14GetChar

This function fetches a single character from the 1401, waiting (with timeouts) for a character if one is not available, and places it in the variable pointed to by pcChar. Returns zero or an error code.

```
U14API(short) U14GetChar(short hand, char *pcChar);
```

U14Stat1401

This function is used to check for characters waiting to be read from the 1401.

```
U14API(short) U14Stat1401(short hand);
```

The return value is greater than zero if there is at least one line of pending input from the 1401 and zero if not. A negative error code may also be returned. The U14Stat1401 function can be used to allow other host activities while we wait for the 1401 to complete a task. The trick is to get the 1401 to send some input to the host after it has finished the task so that U14Stat1401 can be used to detect it. For example:

```
U14LONG alErr[2];                               /* to store our results */
U14SendString(hand, "ADCMEM,F,2,0,1000,0,0,CT,10,10;ERR;");
while (!U14Stat1401(hand))                       /* while no response .. */
    DoIdle();                                     /* ..do some idle operation */
U14LongsFrom1401(hand, (long FAR *)&alErr,2);    /* get errors */
```

In this example we have asked the 1401 to wait for a trigger, then to sample some data and finally to return the error status. As the ADCMEM command is run in dedicated mode using the F option there is no response from the ERR command until the sampling is over. In this case we make use of this waiting time to perform some idle time routines.

U14LineCount

This function returns a count of the current number of lines waiting in the input buffer to be read or a negative error code.

```
U14API(short) U14LineCount(short hand);
```

U14CharCount

This function returns a count of the current number of characters waiting in the input buffer to be read or a negative error code.

```
U14API(short) U14CharCount(short hand);
```

U14SetTimeout

Set the timeout period when waiting to send or receive characters from the 1401. The default timeout period is 3 seconds. There is no return value.

```
U14API(void) U14SetTimeout(short hand, int lTimeOut);
```

`lTimeOut` The required timeout, in milliseconds.

U14GetTimeout

This returns the current timeout period in milliseconds or an error code for a bad handle.

```
U14API(int) U14GetTimeout(short hand);
```

U14WhenToTimeOut

This function returns the time, in system ticks, at which the 1401 will time out. It does this by getting the current time, in system ticks, and adding the timeout period currently set. This function is used internally by the Use1401 functions and is made available for use elsewhere. Use this function with `U14PassedTime`.

```
U14API(int) U14WhenToTimeOut(short hand);
```

U14PassedTime

This function returns non-zero if the time provided (presumed returned by `U14WhenToTimeOut`), has been passed, otherwise zero. This function cannot return an error.

```
U14API(short) U14PassedTime(long lTime);
```

U14OutBufSpace

This function returns the number of characters of space available in the output buffer or a negative error code. A return value of 0 means no room is available in the output buffer.

```
U14API(short) U14OutBufSpace(short hand);
```

U14KillIO1401

This function flushes out the character input and output buffers in the device driver. This is not generally required, except when attempting to recover from an error condition.

```
U14API(short) U14KillIO1401(short hand);
```

Command loading functions

These functions are concerned with the loading of commands into the 1401. Please note that the `U14LdCmd` function is provided for special purposes only; normal applications should use the `U14Ld` function.

U14Ld

This function loads a list of commands into the 1401 if they are not already loaded. It automatically adds the correct file extension (`.CMD`, `.GXC`, `.ARM`, `.ARN` and so on) to the commands in the list depending on the type of 1401 connected. This function uses block transfers (below) to transfer the command data into the 1401. Block transfer area zero is used, so this function will fail if area zero is already in use.

```
U14API(long) U14Ld(short hand, const char *v1, const char *str);
```

`v1` This is either 0 or set to "" (meaning ignore this argument) or is a pointer to a string holding a directory path to be searched for 1401 commands. If the path does not end with a path separator (`\` or `:`), a `\` for Windows or a `/` for other systems is appended.

`command` This is a text string holding the list of 1401 commands to load. The command names are separated by commas and may not include any spaces. Upper case is preferred for command names for compatibility with very old 1401s.

The function searches for the commands in a number of directories in turn:

- The directory specified in the `v1` parameter, if non-blank.
- The 1401 directory in the application directory
- The directory specified in the `1401DIR` environment variable, if it exists.
- The `\1401` directory on the current drive in Windows or in `/var/1401` in Linux.

The function returns an error code indicating the success or failure of the operation. The error code is formed from the command number (starting at 1) in the list in the high word and a normal Use1401 error code in the low word. For example:

```
short sCmd, sErr;
long lRetVal;
lRetVal = U14Ld(hand, "", "ADCMEM,SS2");          /* load commands */
sErr = (short)(lRetVal & 0xFFFF);                 /* get error code */
if (sErr != U14ERR_NOERROR)                        /* see if error occurred */
{
    sCmd = (short)(lRetVal>>16);                  /* index of failed command */
    printf("Command #%d of ADCMEM,SS2 failed to load\n");
}
```

U14LdCmd

This function loads a single command into the 1401, even if it is already loaded. It returns the normal error codes. There are few occasions when this is required, so we suggest that the `U14Ld` function is used in all application software. This uses block transfers to move the command data into the 1401. Block transfer area zero is used, so this function will fail if area zero is already in use.

```
U14API(short) U14LdCmd(short hand, const char *command);
```

`command` This is the name of the file to be loaded. If there is no `.CMD`, `.GXC`, `.ARM`, `.ARN` etc. on the end of the name, the appropriate file extension is added. See the `U14Ld` documentation for details of where the function looks for the command if a full path is not supplied.

Block transfer functions

These functions are all concerned with the transfer of blocks of data between the 1401 and the host computer. These transfers normally use DMA mechanisms, but can operate using programmed I/O. In general, application programmers will only require the `U14ToHost` and `U14To1401` functions, the rest of the block transfer functions are only needed for more demanding programs.

U14ToHost & U14To1401

These two functions are used to pass blocks of data between memory buffers in the host computer and the 1401. The functions call `U14SetTransArea` and `U14UnSetTransfer` as required. Block transfer area number zero is used, so these functions will fail if area zero is already in use.

```
U14API(short) U14ToHost(short hand, char *pHost, DWORD dwSz,
                        DWORD dwA1401, short eSz);
```

```
U14API(short) U14To1401(short hand, const char *pHost, DWORD dwSz,
                        DWORD dwA1401, short eSz);
```

pHost A pointer to the host buffer for the transfer. This region is where the data is put by `U14ToHost` or taken from by `U14To1401`.

dwSz The number of data bytes to transfer. It is crucial that this number does not exceed the size of the buffer pointed at by `lpHost`. If it does a memory protection error will probably occur, if a protection error does not occur the transfer may write all over memory which you do not intend to change. The maximum allowed transfer size is 64 Kbytes.

dwA1401 The address of the start of the 1401 buffer. This is given as a `DWORD` to allow access to the full address range in the 1401.

eSz This parameter was used when the host was a big-endian machine and should be set to 0 as supported machines are all little-endian. This defines what sort of data is to be transferred between the 1401 and the host machine.

The functions return a negative error code if there is a problem and `U14ERR_NOERROR` if all is successful. For example, to collect a buffer of 1024 two-byte data points from the start of the 1401 memory into an array:

```
short asBuffer[1024]; /* buffer for our data */
short sErr = U14ToHost(hand, (char*)asBuffer, 2048, 0, 0);
```

U14SetTransArea

This function prepares an area of memory in the host computer for linear block transfers between the 1401 and the host computer. If necessary, the memory is locked so that it cannot be moved. The function returns an error code.

```
U14API(short) U14SetTransArea(short hand, WORD wArea,
                              void *pBuff, DWORD dwLength, short eSz);
```

wArea The number of the transfer area to set up. This must be a value in the range 0 to 7 except for a standard 1401 which only supports area 0. Very old device driver did not support multiple areas, so if in doubt, you should check the `U14TransferFlags` result to see if multiple transfer areas can be used.

pBuff The address of the buffer that data is transferred from or into.

dwLength The length of the transfer buffer, in bytes. The maximum possible is 1 MB, but older device drivers and versions of `Use1401` have a limit of 256 KB.

eSz The size of the data items being transferred. This is only needed when the host is big-endian, as it is used to set the byte swapping, otherwise 0 is set.

This function is called when you use a 1401 command that transfers binary data to or from host memory such as `TOHOST`, `TO1401` or `CLOAD`. The `U14ToHost`, `U14To1401` and `U14Ld` functions handle the common cases of block transfer and call `U14SetTransArea` internally. The only command you are likely to need to use this for is `PERI32`, which transfers and ‘unwraps’ binary data from it’s circular buffer and will need you to set up a transfer area. Specialised commands, such as those used by `Spike2` also make use of multiple transfer areas.

An example of the hard way to use `TOHOST` illustrates the use of `U14SetTransArea`:

```
char acBuff[10000];           /* buffer for our data */
U14LONG er[2];               /* for decoding 1401 replies */
short sErr=U14SetTransArea(hand, 0, &acBuf, 10000, 0);
if (sErr == U14ERR_NOERROR)   /* no error */
{
    U14SendString(hand, "TOHOST,0,10000,0;ERR;");
    sErr = U14LongsFrom1401(hand, er, 2);
    U14UnSetTransfer(hand, 0); /* free area now */
}
else
    ... here for SetTransArea error ...
```

ESZBYTES, ESZWORDS, ESZLONGS

These constants are defined for use in block transfer routines where it is necessary to define the size of an element as the host is big-endian and byte swapping is required. If you are using complex commands that transfer data structures, all the elements of the data structure must be the same size for the byte swapping to work correctly.

```
#define ESZBYTES 0           /* BYTE element size value (1 byte) */
#define ESZWORDS 1          /* WORD element size value (2 bytes) */
#define ESZLONGS 2          /* long element size value (4 bytes) */
#define ESZUNKNOWN 0        /* unknown element size value */
```

U14UnSetTransfer

This function releases a transfer area so that it is available for use elsewhere. If it is not used then further `U14SetTransArea` calls with the same area number will fail. It also unlocks the memory locked by `U14SetTransArea`. Locking memory reduces system performance, therefore it is important to only set up a transfer area when it is needed and close it down once it has been used. See above for an example of use.

```
U14API(short) U14UnSetTransfer(short hand, WORD wArea);
```

wArea The number of the transfer area to release in the range 0 to 7. It must be the same as used in the corresponding `U14SetTransArea` call.

U14BlkTransState

Returns 1 if a block transfer is in progress, 0 if not or a negative error code. In progress means that the device driver is busy moving data; there should never be any reason to use this function.

```
U14API(short) U14BlkTransState(short hand);
```

U14SetCircular

This function sets up a transfer area for circular transfers and is an alternative to the U14SetTransferArea command. The transfer type used depends on the code run in the 1401; at the time of writing, only Spike2 makes use of circular transfers, and only in the to host direction. The circular transfer method allows 1401 code to amalgamate a large number of small transfers into a smaller number of larger transfers. This is particularly important when USB is used as there is a significant transfer set up overhead.

```
U14API(short) U14SetCircular(short hand, WORD wArea,  
                             BOOL bToHost, void *pBuff, DWORD dwLength);
```

wArea The transfer area to use in the range 0-7, as for U14SetTransferArea.

bToHost The transfer direction. At the time of writing this must be TRUE as circular transfers to the 1401 are not implemented.

pBuff The memory block that is to be used as a transfer buffer. This should be at least DWORD aligned.

dwLength The size of the memory buffer, in bytes.

Returns U14ERR_NOERR if all was well or a negative error code.

You can cancel an area with U14UnSetTransfer. Once you have used this function, you will likely use U14GetCircBlk() and U14FreeCircBlock() to get data to process and U14SetTransferEvent() to detect when new data is available.

U14GetCircBlock

This function returns the size of and the offset to the circular data that is available. Once you have used the data, you must call U14FreeCircBlock() to notify the library that you are done with the data and to collect the next block. Although you can call this in a loop, it can be more efficient to use U14WaitTransferEvent() with a thread so that you make efficient use of the CPU to collect data.

```
U14API(int) U14GetCircBlk(short hand, WORD wArea, DWORD *pdwOffs)
```

wArea The transfer area to use in the range 0-7, as for U14SetTransferArea.

pdwOffs This points at a DWORD that is set to the offset in the transfer area of the start of the data. This offset will always be DWORD aligned.

Returns The number of bytes that are available (or 0 if none) or a negative error code.

U14FreeCircBlock

This function notified the library that you are done with a block of memory that was returned by a call to U14GetCircBlock or U14FreeCircBlock. It also returns the next block (if any) to be processed.

```
U14API(int) U14FreeCircBlk(short hand, WORD wArea,  
                           DWORD dwOffs, DWORD dwSize, DWORD *pdwOffs);
```

wArea The transfer area to use in the range 0-7, as for U14SetTransferArea.

dwOffs The offset as returned by this function or U14GetCircBlock to the memory to be released.

dwSize The size of the memory to be released. This will usually be the same as the last returned size, but can be less as long as the size is a multiple of 4 bytes.

pdwOffs This points at a DWORD that is set to the offset in the transfer area of the start of the next block of data.

Returns The number of bytes that are available in the next block.

U14SetTransferEvent

This sets a transfer area so that an application can either test for, or wait efficiently for a linear data transfer to occur within a specified sensitive area of a transfer area or for a circular transfer to have a certain number of bytes of data available. Drivers that support this return `U14TF_NOTIFY` from the `U14TransferFlags` function. Only very old drivers cannot handle this. Once a transfer event is set, the `U14WaitTransferEvent()` and `U14TestTransferEvent()` routines become available to determine if a particular transfer is set. Only one transfer event can be set for each region.

This command works in two distinct ways, depending on what type of transfer (circular or linear) is being done in the area. For circular transfers, you can detect when a certain number of bytes are available. For linear transfers, you can detect when a transfer has occurred in the specified sensitive area.

```
U14API(short) U14SetTransferEvent(short hand, WORD wArea,
                                BOOL bEvent, BOOL bToHost,
                                DWORD dwStart, DWORD dwLength);
```

wArea A transfer area in the range 0-7 that has been set by `U14SetTransferArea`, otherwise `U14ERR_NOTSET` is returned.

bEvent In Windows, notifications are managed using event objects. Set this `TRUE` to associate an event with an area, set it `FALSE` to remove the event. In Linux, we use a different mechanism, but you should still set this flag as if events were being created or removed.

bToHost Set this `TRUE` to set events on a transfer to the host, `FALSE` for events on a transfer to the 1401. If the area is set for a circular transfer, this must be `TRUE`.

dwStart This is not used for circular transfer areas where it should be set to 0. For a linear area, it sets the offset to the start of sensitive area.

dwLength For a circular transfer, this sets the number of bytes that should be available for the testing and waiting functions to succeed. For a linear transfer, this sets the size of the sensitive area.

Returns The function returns 1 if an event is available for testing, 0 if no event is available or a negative error code if there was a problem.

Use `U14SetTransferEvent(hand, wArea, 0, 0, 0, 0);` to clear a transfer event.

U14TestTransferEvent

This functions returns 1 if an event set by `U14SetTransferEvent` has occurred since the last test, 0 if it has not, or a negative error code (area not set, for example).

```
U14API(int) U14TestTransferEvent(short hand, WORD wArea);
```

wArea A transfer area in the range 0-7 that has been set by `U14SetTransferEvent`.

U14WaitTransferEvent

This function waits until an event set by `U14SetTransferEvent` has occurred, or a timer expires. Waiting is done by suspending the current thread, so the timeout sets a minimum period until the thread resumes. It is much more efficient (in terms of CPU usage) to use this than to continuously poll using `U14TestTransferEvent`.

```
U14API(int) U14WaitTransferEvent(short hand, WORD wArea, int ms);
```

wArea A transfer area in the range 0-7 that has been set by `U14SetTransferEvent`.

ms If no event occurs after this time, stop waiting.

Returns 1 if timed out, 0 (`U14ERR_NOERROR`) if the event occurred or a negative error.

Test and diagnostic functions

The following structures and functions are used for testing the 1401 in various ways. They are of little use to normal applications software and should be avoided by non-CED programmers.

BYTE_SIZE, WORD_SIZE & LONG_SIZE

These constants are defined for use in the diagnostics functions where it is necessary to specify the width of a data read or write. Note that, for the standard 1401, the size is ignored as all reads and writes are 8 bit. Note also that the micro1401 cannot do word accesses and uses 4-byte access instead

```
#define BYTE_SIZE 1      /* 8-bit data access      */
#define WORD_SIZE 2      /* 16-bit data access     */
#define LONG_SIZE 3      /* 32-bit data access     */
```

U14StartSelfTest

This starts a self-test pass in the 1401. It returns zero or a negative error code.

```
U14API(short) U14StartSelfTest(short hand);
```

U14CheckSelfTest

This function checks to see if the self-test pass started by U14StartSelfTest has returned any errors, timed out, or finished. The function result is zero or an error code.

```
U14API(short) U14CheckSelfTest(short hand, U14LONG *pData);
```

pData This points to an array of 3 longs that will hold the returned information on the progress of the self-test. The first long holds information on the status; 0 for still waiting, -1 for self-test finished, -2 for self-test timeout, all other values are self-test error codes. The other two longs hold information on the self-test error, if any.

U14GetTransfer

This function returns information about a (previously set up) transfer area. The function result is zero or a negative error code.

```
U14API(short) U14GetTransfer(short hand, TGET_TX_BLOCK *pBlock);
```

pBlock Points at a GET_TX_BLOCK to be updated with transfer area information. See the system specific section, below, for details of TGET_TX_BLOCK.

The **size** field of the structure should be set by the caller to the transfer area number for which information is required. The routine then fills in the structure with the current settings for that table entry. These values are undefined if the entry has never been used.

U14BaseAddr1401

This function returns the base address of the 1401 interface card registers. This is normally 0x300 for the ISA interface card, can be anything for the PCI card, is -1 for the USB interface. There is no reason to use this in modern software.

```
U14API(long) U14BaseAddr1401(short hand);
```

U14Grab1401

This function causes the 1401 device driver to grab control of the 1401. This function must be called before any of the diagnostics functions described below are used.

```
U14API(short) U14Grab1401(short hand);
```

U14Free1401

This function allows the 1401 to free run again, cancelling the effect of U14Grab1401. You must call this function to undo the effects of U14Grab1401.

```
U14API(short) U14Free1401(short hand);
```

U14Poke1401

This diagnostic function writes specified data to an address inside the 1401. This function can only be used when the 1401 has been grabbed by the device driver.

```
U14API(short) U14Poke1401(short hand, DWORD lAddr, DWORD lValue,  
                           int lSize, int lRepeats);
```

lAddr The address, within the 1401 address space, to be written to. Please note that you need to know about the internals of the 1401 in order to choose addresses that are safe to write to.

lValue The value to write to the address.

lSize The size of the write access. This should be one of `BYTE_SIZE`, `WORD_SIZE` or `LONG_SIZE` as appropriate. Note that not all types of 1401 support all sizes of data access.

lRepeats The number of times to write to the location. To write continuously until `U14StopDebugLoop` is called, use a repeat count of zero.

U14Peek1401

This diagnostic function reads from a specified address inside the 1401, the data read is saved internally and can be retrieved using `U14GetDebugData`. This function can only be used when the 1401 has been grabbed by the device driver.

```
U14API(short) U14Peek1401(short hand, DWORD lAddr, DWORD lSize,  
                           int lRepeats);
```

lAddr The address, within the 1401 address space, to be read from. You will need to be familiar with the 1401 internals in order to choose a suitable address.

lSize The size of the read access. This should be one of `BYTE_SIZE`, `WORD_SIZE` or `LONG_SIZE` as appropriate. Note that not all types of 1401 support all sizes of data access.

lRepeats The number of times to read from the location. To read continuously until `U14StopDebugLoop` is called, use a repeat count of zero.

U14Ramp1401

This diagnostic function writes a data ramp to an address inside the 1401. This function can only be used when the 1401 has been grabbed by the device driver.

```
U14API(short) U14Ramp1401(short hand, DWORD lAddr, DWORD lDef,  
                           DWORD lEna, int lSize, int lRepeats);
```

lAddr The address, within the 1401 address space, to be written to. Please note that you need to know about the internals of the 1401 in order to choose addresses that are safe to write to.

lDef The default value to write to the address. This is the initial start of the data ramp and sets the state of any disabled bits.

- lEna** A bit mask enabling bits to be ramped. Bits in the mask that are set will have the ramping data written to them, bits that are clear will have only the appropriate default value written.
- lSize** The size of the write access. This should be one of `BYTE_SIZE`, `WORD_SIZE` or `LONG_SIZE` as appropriate. Note that not all types of 1401 support all sizes of data access. For `WORD_SIZE` and `LONG_SIZE` ramps, all of the bytes are ramped simultaneously.
- lRepeats** The number of times to write the ramp to the location, a value of one selects a single 256 value ramp. To write continuously until `U14StopDebugLoop` is called, use a repeat count of zero.

U14RampAddr

This diagnostic function reads from a ramping address inside the 1401. The individual bytes within the address are ramped simultaneously. This function can only be used when the 1401 has been grabbed by the device driver.

```
U14API(short) U14RampAddr(short hand, DWORD lDef, DWORD lEna,
                          int lSize, int lRepeats);
```

- lDef** The default address value to read from. This is the initial start of the address ramp and sets the state of any disabled bits in the address.
- lEna** A bit mask enabling address bits to be ramped. Bits in the mask that are set will be used as part of the ramping address, bits that are clear will stick at the default value.
- lSize** The size of the read access. This should be one of `BYTE_SIZE`, `WORD_SIZE` or `LONG_SIZE` as appropriate. Note that not all types of 1401 support all sizes of data access.
- lRepeats** The number of times to read the ramp, a value of one selects a single 256 value ramp. To read continuously until `U14StopDebugLoop` is called, use a repeat count of zero.

U14StopDebugLoop

This function causes any continuous debug loops (reads or writes set up with a repeat count of zero) to terminate.

```
U14API(short) U14StopDebugLoop(short hand);
```

U14GetDebugData

This function returns the result of the last use of `U14Peek1401`, if the 1401 has been set to peek continuously then `U14StopDebugLoop` must be called first.

```
U14API(short) U14GetDebugData(short hand, U14LONG *plValue);
```

plValue This points to a long that will be updated with the value read from the 1401.

Windows specifics

Overview The Windows version of the Use1401 library has some extra features which are described below. The documentation gives C/C++ function prototypes only, though use from other languages would be equivalent.

DLL behaviour The Use1401 code is designed to be used as a DLL. The unit includes the `DllMain` function required by Windows for it to use the unit as a DLL:

```
int APIENTRY DllMain(HANDLE hInst, DWORD dwReason,
                    LPVOID lpReserved);
```

The `DllMain` code finds the windows version and sorts out how to call the device driver.

Direct calls to the driver The device driver may be called directly by using the following functions, which are not exported by the DLL. These functions should not be used except for development.

```
U14API(short) U14Status1401(short h, long lCode, TCSBLOCK* pBlk);
U14API(short) U14Control1401(short h, long lCode, TCSBLOCK* pBlk);
```

The first parameter is the handle (or 1401 number for Open), the next parameter is a function code which is passed to the driver. All the standard function codes are defined in `USE1401.H`. The final parameter is a pointer to a control/status block used to hold information relevant to this function. Interfaces that are shared between the library and the device driver are declared in the header file `use14_ioc.h`.

TGET_TX_BLOCK This structure is used by the `U14GetTransfer` function to return information about an area set up for block transfers, it allows debug and test software to see the behaviour of the Windows device drivers. Note that the Linux version of this structure is different.

```
C/C++ #define GET_TX_MAXENTRIES 257 /* # blocks */

typedef struct /* used for U14_GetTransfer results */
{
    long    physical;
    long    size;
} TXENTRY;

typedef struct TGetTxBlock /* used for U14_GetTransfer results */
{
    long    size; /* set to area number before calling */
    long    linear;
    short   seg;
    short   reserved;
    short   avail;
    short   used;
    TXENTRY entries[GET_TX_MAXENTRIES]; /* mapped block info */
} TGET_TX_BLOCK;
```

```
Borland Pascal/Delphi CONST GET_TX_MAXENTRIES = 257; { # blocks }

TYPE TXEntry=RECORD { for U14_GetTransfer results }
    physical:LONGINT; { Info on one mapped block }
    size:LONGINT;
END;
```



```
TGet_TX_Block=RECORD    { used for U14_GetTransfer results }
    size:LONGINT;        { matches structure in VXD }
    linear:LONGINT;
    seg:INTEGER;
    reserved:INTEGER;
    avail:INTEGER;
    used:INTEGER;
    entries: ARRAY[0..GET_TX_MAXENTRIES-1] OF TXEntry;
END;
```

Linux specifics

Overview This section is preserved in case we ever revive Linux support, which would entail getting the Linux USB driver working again. We last had this working in 2012. At that time it was quite difficult to keep up with the changes to Linux USB support; it is likely that this has stabilized in the intervening years.

Under Linux, the only supported 1401 driver was for USB. The library routines are pretty much identical to Windows, the only exception being in the `U14GetTransfer()` function where the `TGET_TX_BLOCK` function differs between Windows and Linux due to 64-bit memory addressing differences.

TGET_TX_BLOCK This structure is used by the `U14GetTransfer` function to return information about an area set up for block transfers, it allows debug and test software to see the behaviour of the device drivers.

```
#define GET_TX_MAXENTRIES    257 /* # blocks */

typedef struct              /* used for U14GetTransfer results */
{                          /* Info on a single mapped block */
    long long physical;
    long      size;
} TXENTRY;

typedef struct TGetTxBlock
{
    long long linear; /* linear address */
    long      size;   /* set to area number before calling */
    short     seg;    /* segment of the address for Win16 */
    short     reserved;
    short     avail;  /* number of available entries */
    short     used;   /* number of used entries */
    TXENTRY   entries[GET_TX_MAXENTRIES]; /* block info */
} TGET_TX_BLOCK;
```

There are also differences in where command files are stored.

Direct calls to the driver The header file `ced_ioctl.h` holds definitions of structures and macros that are used to communicate between the library and the device driver.

Delphi & Borland Pascal specifics

Introduction The file `USE1432.PAS` declares the Use1432 DLL as a unit for Delphi 2 software. Runtime linkage to the DLL is automatic and does not require any coding. All functions and error codes defined have the prefix U14 to indicate their origin.

The documentation for the Use1401 functions, and many other aspects of the library, are given as part of the C/C++ documentation because use from Borland Pascal or Delphi is simply a matter of getting access to the functions in the DLL, the actual functions used, the function parameters and the behaviour of the functions are identical for all languages.

Types and records A number of types and structures are defined in `USE1401.PAS` and `USE1432.PAS` for use by the various routines. Many of these are unimportant or are provided for specialised purposes, the C documentation gives more details of these; the more important ones are documented below:

DWORD This type defines an (assumed) unsigned 4 byte integer, for general use in the interface and for similarity to the underlying C code.

```
TYPE DWORD = LONGINT;
```

TpNums This type defines a pointer to an array of `LONGINT`s, this array is assumed to be of an unknown length. This type is used as an argument to the `U14LongsFrom1401` function and other similar functions.

Visual Basic specifics

Introduction

The Use1401 library can be used from Visual BASIC Windows programs. 16-bit Visual BASIC programs produced by Visual-BASIC versions 3 and 4 use the 16-bit DLL USE1401.DLL (described in separate documentation), while 32-bit programs produced using Visual BASIC version 5 can use the 32-bit DLL use1432.dll. The use1432.tdl file defines types and functions and you will need to add this, for example using the Tools menu References... command when using Visual Basic.

The documentation for the Use1401 functions, and many other aspects of the library, are given as part of the C/C++ documentation because use from Visual BASIC is simply a matter of getting access to the functions in the DLL, the actual functions used, the function parameters and the behaviour of the functions are identical for all languages. Note that the use1432.dll library must be available in either the same folder as the application or in one of the system locations for DLLs, or it must be in a folder in the PATH environment variable.

Get1401String

This function is provided in the Visual BASIC support to provide a more convenient Visual Basic string read mechanism. It ensures that the string passed to the U14GetString routine is long enough to receive any string that the 1401 may return.

```
Function Get1401String (ByVal handle1401 As Integer,  
                      strP As String, ByVal maxlen As Integer) As Integer
```

This function calls U14GetString and returns the same error codes. It reads up to maxlen characters from the 1401 and times out if there are none ready. strP may safely be empty before you call this function. For example:

```
Dim strP As String  
strP = ""  
If Get1401String (hand1401, strP, 20) Then 'read up to 20 characters  
                                           'we have characters in strP  
End If
```

Getting started

We suggest that you work through the first few chapters of the Visual Basic Programmer's Guide until you can find your way around the Project. Chapters 1 to 7 introduce properties, code, controls and menu design. Once you are familiar with the Visual Basic environment you can add the Use1401 support library and add code to access the 1401.

There are example projects CED_VB01, CED_VB02, and CED_VB03 demonstrating the use of Use1401 from Visual BASIC. The files for these examples have the extensions .FRM, .TXT and .MAK and there is also a text file README.DOC describing them. You could either create a new project or use code from one or more of these example projects as a starting point, copying code from one project to another.

Python specifics

Introduction The Use1401 library can be used from Python in Windows from the py1401 library. This is described in detail in the printable `py1401.pdf` file and in the interactive Help file, `py1401.chm`.

The following example opens the a 1401, loads a 1401 command, starts sampling, waits for 500 data points to have been collected, collects the data and prints some statistics about it and finally, closes the 1401.

```
import numpy as np          # we use numpy for arrays in this example
import pyl401
u14 = pyl401.create();      # make instance to talk to a 1401
if not u14.open():
    print('Failed to open the 1401: ', u14.error());
    exit();

if u14.ld('adcmem', 'c:/1401'):
    print('Failed to load commands: ', u14.error());
    exit();

if not u14.write('adcmem,i,2,0,10000,0,0,c,100,100;'): #100 Hz
    print('Go failed: ', u14.error());
    exit();

points = 500;

n = 0;
while n < points*2:         # 2 bytes per data point
    u14.write('adcmem,p;');
    pos = u14.readInts();
    n = pos[0];
    print(n);

data = np.zeros([points], dtype=np.int16);
u14.tohost(data, 0);

print(u14.error());
print(data);
print(np.mean(data), np.std(data), np.max(data), np.min(data));

u14.write('adcmem,k;'); # stop sampling
u14.close();
```

You are referred to the `py1401.pdf` and `py1401.chm` files for further information.

Appendix 1: Listing of use1401.h

```
#ifndef __USE1401_H__
#define __USE1401_H__
#include "machine.h"

/// Some definitions to make things compatible
#ifdef _IS_WINDOWS_
#ifndef U14_NOT_DLL
#ifdef DLL_USE1401
#define U14API(retType) retType DllExport __stdcall
#else
#define U14API(retType) retType DllImport __stdcall
#endif
#endif
#endif

#define U14ERRBASE -500
#define U14LONG long
#endif

#ifdef LINUX
#define U14ERRBASE -1000
#define U14LONG int
#endif

#ifdef _QT
#ifndef U14_NOT_DLL
#undef U14API
#define U14API(retType) retType __declspec(dllimport) __stdcall
#endif
#undef U14LONG
#define U14LONG int
#endif

#ifdef U14API
#define U14API(retType) retType
#endif

#ifdef U14LONG
#define U14LONG long
#endif

/// Error codes: We need them here as user space can see them.
#define U14ERR_NOERROR 0 // no problems

/// Device error codes, but these don't need to be extended - a succession is assumed
#define U14ERR_STD 4 // standard 1401 connected
#define U14ERR_U1401 5 // u1401 connected
#define U14ERR_PLUS 6 // 1401 plus connected
#define U14ERR_POWER 7 // Power1401 connected
#define U14ERR_U14012 8 // u1401 mkII connected
#define U14ERR_POWER2 9
#define U14ERR_U14013 10
#define U14ERR_POWER3 11

/// NBNB Error numbers need shifting as some linux error codes start at 512
#define U14ERR(n) (n+U14ERRBASE)
#define U14ERR_OFF U14ERR(0) /* 1401 there but switched off */
#define U14ERR_NC U14ERR(-1) /* 1401 not connected */
#define U14ERR_ILL U14ERR(-2) /* if present it is ill */
#define U14ERR_NOIF U14ERR(-3) /* I/F card missing */
#define U14ERR_TIME U14ERR(-4) /* 1401 failed to come ready */
#define U14ERR_BADSW U14ERR(-5) /* I/F card bad switches */
```

```

#define U14ERR_PTIME          U14ERR(-6)      /* 1401plus failed to come ready */
#define U14ERR_NOINT          U14ERR(-7)      /* couldn't grab the int vector */
#define U14ERR_INUSE          U14ERR(-8)      /* 1401 is already in use */
#define U14ERR_NODMA          U14ERR(-9)      /* couldn't get DMA channel */
#define U14ERR_BADHAND        U14ERR(-10)     /* handle provided was bad */
#define U14ERR_BAD1401NUM     U14ERR(-11)     /* 1401 number provided was bad */

#define U14ERR_NO_SUCH_FN      U14ERR(-20)     /* no such function */
#define U14ERR_NO_SUCH_SUBFN   U14ERR(-21)     /* no such sub function */
#define U14ERR_NOOUT          U14ERR(-22)     /* no room in output buffer */
#define U14ERR_NOIN           U14ERR(-23)     /* no input in buffer */
#define U14ERR_STRLIN         U14ERR(-24)     /* string longer than buffer */
#define U14ERR_ERR_STRLIN     U14ERR(-24)     /* string longer than buffer */
#define U14ERR_LOCKFAIL       U14ERR(-25)     /* failed to lock memory */
#define U14ERR_UNLOCKFAIL     U14ERR(-26)     /* failed to unlock memory */
#define U14ERR_ALREADYSET     U14ERR(-27)     /* area already set up */
#define U14ERR_NOTSET         U14ERR(-28)     /* area not set up */
#define U14ERR_LOCKERR        U14ERR(-29)     /* illegal area number */
#define U14ERR_FAIL           U14ERR(-30)     /* we failed for some other reason*/

#define U14ERR_NOFILE         U14ERR(-40)     /* command file not found */
#define U14ERR_READERR        U14ERR(-41)     /* error reading command file */
#define U14ERR_UNKNOWN        U14ERR(-42)     /* unknown command */
#define U14ERR_HOSTSPACE      U14ERR(-43)     /* not enough host space to load */
#define U14ERR_LOCKERR        U14ERR(-44)     /* could not lock resource/command*/
#define U14ERR_CLOADERR       U14ERR(-45)     /* CLOAD command failed */

#define U14ERR_TOXXERR        U14ERR(-60)     /* tohost/1401 failed */
#define U14ERR_NO386ENH       U14ERR(-80)     /* not 386 enhanced mode */
#define U14ERR_NO1401DRIV     U14ERR(-81)     /* no device driver */
#define U14ERR_DRIVTOOOLD     U14ERR(-82)     /* device driver too old */

#define U14ERR_TIMEOUT        U14ERR(-90)     /* timeout occurred */

#define U14ERR_BUFF_SMALL     U14ERR(-100)    /* buffer for getstring too small */
#define U14ERR_CBALREADY      U14ERR(-101)    /* there is already a callback */
#define U14ERR_BADDEREG       U14ERR(-102)    /* bad parameter to deregcallback */
#define U14ERR_NOMEMORY       U14ERR(-103)    /* no memory for allocation */

#define U14ERR_DRIVCOMMS      U14ERR(-110)    /* failed talking to driver */
#define U14ERR_OUTOFMEMORY     U14ERR(-111)    /* needed memory and couldnt get it*/

/// 1401 type codes.
#define U14TYPE1401           0               /* standard 1401 */
#define U14TYPEPLUS           1               /* 1401 plus */
#define U14TYPEU1401          2               /* u1401 */
#define U14TYPEPOWER           3               /* power1401 */
#define U14TYPEU14012         4               /* u1401 mk II */
#define U14TYPEPOWER2         5               /* power1401 mk II */
#define U14TYPEU14013         6               /* u1401-3 */
#define U14TYPEPOWER3         7               /* power1401-3 */
#define U14TYPEUNKNOWN        -1              /* dont know */

/// Transfer flags to allow driver capabilities to be interrogated

/// Constants for transfer flags
#define U14TF_USEDMA           1               /* Transfer flag for use DMA */
#define U14TF_MULTIA          2               /* Transfer flag for multi areas */
#define U14TF_FIFO            4               /* for FIFO interface card */
#define U14TF_USB2            8               /* for USB2 interface and 1401 */
#define U14TF_NOTIFY          16              /* for event notifications */

```

```
#define U14TF_SHORT          32          /* for PCI can short cycle      */
#define U14TF_PCI2           64          /* for new PCI card 1401-70    */
#define U14TF_CIRCTH         128         /* Circular-mode to host       */
#define U14TF_DIAG           256         /* Diagnostics/debug functions */
#define U14TF_CIRC14         512         /* Circular-mode to 1401       */

/// Definitions of element sizes for DMA transfers - to allow byte-swapping
#define ESZBYTES             0           /* BYTE element size value     */
#define ESZWORDS             1           /* WORD element size value     */
#define ESZLONGS             2           /* long element size value     */
#define ESZUNKNOWN           0           /* unknown element size value  */

/// These define required access types for the debug/diagnostic function
#define BYTE_SIZE            1           /* 8-bit access                 */
#define WORD_SIZE            2           /* 16-bit access                */
#define LONG_SIZE            3           /* 32-bit access                */

/// Stuff used by U14_GetTransfer
#define GET_TX_MAXENTRIES    257         /* (max length / page size + 1) */

#ifdef _IS_WINDOWS_
#pragma pack(1)

typedef struct                /* used for U14_GetTransfer results */
{                             /* Info on a single mapped block */
    U14LONG physical;
    U14LONG size;
} TXENTRY;

typedef struct TGetTxBlock    /* used for U14_GetTransfer results */
{                             /* matches structure in VXD */
    U14LONG size;
    U14LONG linear;
    short seg;
    short reserved;
    short avail;              /* number of available entries */
    short used;               /* number of used entries */
    TXENTRY entries[GET_TX_MAXENTRIES]; /* Array of mapped block info */
} TGET_TX_BLOCK;

typedef TGET_TX_BLOCK *LPGET_TX_BLOCK;

#pragma pack()
#endif

#ifdef LINUX
typedef struct                /* used for U14_GetTransfer results */
{                             /* Info on a single mapped block */
    long long physical;
    long size;
} TXENTRY;

typedef struct TGetTxBlock    /* used for U14_GetTransfer results */
{                             /* matches structure in VXD */
    long long linear;         /* linear address */
    long size;                /* total size of the mapped area, holds id when
called */
    short seg;                /* segment of the address for Win16 */
    short reserved;
    short avail;              /* number of available entries */
    short used;               /* number of used entries */
}
```



```
    TXENTRY  entries[GET_TX_MAXENTRIES]; /* Array of mapped block info */
} TGET_TX_BLOCK;
#endif

#ifdef __cplusplus
extern "C" {
#endif

U14API(int) U14WhenToTimeOut(short hand);    // when to timeout in ms
U14API(short) U14PassedTime(int iTime);      // non-zero if iTime passed

U14API(short) U14LastErrCode(short hand);

U14API(short) U14Open1401(short n1401);
U14API(short) U14Close1401(short hand);
U14API(short) U14Reset1401(short hand);
U14API(short) U14ForceReset(short hand);
U14API(short) U14TypeOf1401(short hand);
U14API(short) U14NameOf1401(short hand, char* pBuf, WORD wMax);

U14API(short) U14Stat1401(short hand);
U14API(short) U14CharCount(short hand);
U14API(short) U14LineCount(short hand);

U14API(short) U14SendString(short hand, const char* pString);
U14API(short) U14GetString(short hand, char* pBuffer, WORD wMaxLen);
U14API(short) U14SendChar(short hand, char cChar);
U14API(short) U14GetChar(short hand, char* pcChar);

U14API(short) U14LdCmd(short hand, const char* command);
U14API(DWORD) U14Ld(short hand, const char* v1, const char* str);

U14API(short) U14SetTransArea(short hand, WORD wArea, void *pvBuff,
                              DWORD dwLength, short eSz);
U14API(short) U14UnSetTransfer(short hand, WORD wArea);
U14API(short) U14SetTransferEvent(short hand, WORD wArea, BOOL bEvent,
                                  BOOL bToHost, DWORD dwStart, DWORD dwLength);
U14API(int) U14TestTransferEvent(short hand, WORD wArea);
U14API(int) U14WaitTransferEvent(short hand, WORD wArea, int msTimeOut);
U14API(short) U14GetTransfer(short hand, TGET_TX_BLOCK *pTransBlock);

U14API(short) U14ToHost(short hand, char* pAddrHost, DWORD dwSize, DWORD dw1401,
                        short eSz);
U14API(short) U14To1401(short hand, const char* pAddrHost, DWORD dwSize, DWORD dw1401,
                        short eSz);

U14API(short) U14SetCircular(short hand, WORD wArea, BOOL bToHost, void *pvBuff,
                             DWORD dwLength);

U14API(int) U14GetCircBlk(short hand, WORD wArea, DWORD *pdwOffs);
U14API(int) U14FreeCircBlk(short hand, WORD wArea, DWORD dwOffs, DWORD dwSize,
                           DWORD *pdwOffs);

U14API(short) U14StrToLongs(const char* pszBuff, U14LONG *palNums, short sMaxLongs);
U14API(short) U14LongsFrom1401(short hand, U14LONG *palBuff, short sMaxLongs);

U14API(void) U14SetTimeout(short hand, int lTimeout);
U14API(int) U14GetTimeout(short hand);
U14API(short) U14OutBufSpace(short hand);
U14API(int) U14BaseAddr1401(short hand);
U14API(int) U14DriverVersion(short hand);
```

```
U14API(int)    U14DriverType(short hand);
U14API(short) U14DriverName(short hand, char* pBuf, WORD wMax);
U14API(short) U14GetUserMemorySize(short hand, DWORD *pMemorySize);
U14API(short) U14KillIO1401(short hand);

U14API(short) U14BlkTransState(short hand);
U14API(short) U14StateOf1401(short hand);

U14API(short) U14Grab1401(short hand);
U14API(short) U14Free1401(short hand);
U14API(short) U14Peek1401(short hand, DWORD dwAddr, int nSize, int nRepeats);
U14API(short) U14Poke1401(short hand, DWORD dwAddr, DWORD dwValue, int nSize, int nRepeats);
U14API(short) U14Ramp1401(short hand, DWORD dwAddr, DWORD dwDef, DWORD dwEnable, int nSize,
int nRepeats);
U14API(short) U14RampAddr(short hand, DWORD dwDef, DWORD dwEnable, int nSize, int nRepeats);
U14API(short) U14StopDebugLoop(short hand);
U14API(short) U14GetDebugData(short hand, U14LONG *pIValue);

U14API(short) U14StartSelfTest(short hand);
U14API(short) U14CheckSelfTest(short hand, U14LONG *pData);
U14API(short) U14TransferFlags(short hand);
U14API(void)  U14GetErrorString(short nErr, char* pStr, WORD wMax);
U14API(int)   U14MonitorRev(short hand);
U14API(void)  U14CloseAll(void);

U14API(short) U14WorkingSet(DWORD dwMinKb, DWORD dwMaxKb);
U14API(int)   U14InitLib(void);

#ifdef __cplusplus
}
#endif

#endif /* End of ifndef __USE1401_H__ */
```

Index

3

32-bit Windows library, 1

B

Base address of card, 17

C

Checking for input, 10
Converting string to longs, 10

D

Direct calls to the driver, 20
Dllmain, 20
Driver type, 8
Driver version, 7

E

Error as string, 7
Error codes, 4, 7

G

Get1401String, 24
Getting a numeric reply, 9

L

Loading commands, 12

M

Memory size, 7
Monitor ROM version, 8

O

Opening the 1401, 5
Output buffer space, 11

R

Reading a string, 9

S

Sending a string, 9
Setting up a block transfer, 13
State of 1401, 7

T

TGET_TX_BLOCK, 20
Timeouts, 11
Transfer in progress check, 14
Transferring data, 13

U

U14API, 3
U14BaseAddr1401, 17
U14BlkTransState, 14
U14CharCount, 11
U14CheckSelfTest, 17
U14Close1401, 6
U14DriverType, 8
U14DriverVersion, 8
U14Free1401, 18
U14FreeCircBlk, 15
U14GetChar, 10
U14GetCircBlk, 15
U14GetDebugData, 19
U14GetErrorString, 7
U14GetString, 9
U14GetTimeout, 11

U14GetTransfer, 17
U14GetUserMemorySize, 7
U14Grab1401, 17
U14KillIO1401, 11
U14LastErrCode, 7
U14Ld, 12
U14LdCmd, 12
U14LineCount, 11
U14LongsFrom1401, 9
U14MonitorRev, 8
U14NameOf1401, 6
U14Open1401, 5
U14OutBufSpace, 11
U14PassedTime, 11
U14Peek1401, 18
U14Poke1401, 18
U14Ramp1401, 18
U14RampAddr, 19
U14Reset1401, 6
U14Reset1401, 6
U14SendChar, 10
U14SendString, 9
U14SetCircular, 15
U14SetTimeout, 11
U14SetTransArea, 13
U14SetTransferEvent, 16
U14StartSelfTest, 17
U14Stat1401, 10
U14StateOf1401, 7
U14StopDebugLoop, 19
U14StrToLongs, 10
U14TestTransferEvent, 16
U14To1401, 13
U14ToHost, 13
U14TransferFlags, 8
U14TypeOf1401, 6
U14UnSetTransfer, 14
U14WaitTransferEvent, 16
U14WhenToTimeout, 11
USE1401.H, 3